SỔ TAY ICPC — BẢN 18 TRANG KHỔ NGANG

Phong cách code: `#include<bits/stdc++.h>`, `using namespace std`, `long long`, `fast IO` | C++17 | Tiếng Việt | Tổng hợp cấu trúc dữ liệu và thuật toán trọng tâm

Mục lục

1 BFS và các dạng đồ thị khó **2**
- 1.1 BFS ma trận có truy vết đường đi . . . 2
- 1.2 BFS 0–1 (cạnh trọng số 0/1) . . . 2
- 1.3 Dijkstra heap + dừng sớm . . . 2
- 1.4 BFS đa nguồn . . . 2
- 1.5 DSU + Kruskal (khó ở nén + rollback tư duy) . . . 2
- 1.6 LCA Binary Lifting . . . 2

2 Fenwick Tree (BIT) **3**
- 2.1 BIT cộng điểm – hỏi prefix/đoạn . . . 3
- 2.2 BIT range-add – point-query và range-add – range-sum . . . 3
- 2.3 BIT tìm chỉ số có prefix $\geq k$ (order statistics) . . . 3
- 2.4 BIT 2D (dạng khó) . . . 3

3 Segment Tree **3**
- 3.1 SegTree lazy cộng – hỏi max/sum . . . 3
- 3.2 Tìm phòng đầu tiên đủ chỗ (Hotel Queries) . . . 4
- 3.3 SegTree lặp (iterative) cho sum/min . . . 4

4 Trie **4**
- 4.1 Trie chuỗi thường (đếm prefix) . . . 4
- 4.2 Binary Trie – XOR lớn nhất (dạng khó) . . . 4
- 4.3 Aho–Corasick (Trie + KMP tập mẫu) . . . 4

5 KMP, Z, Hashing **4**
- 5.1 Hàm prefix π và KMP tìm mẫu . . . 4
- 5.2 Hàm Z . . . 5
- 5.3 Rolling Hash kép . . . 5
- 5.4 Manacher (palindrome dài nhất $O(n)$) . . . 5

6 DP chữ số (Digit DP) **5**

7 DP Bitmask và các dạng khó **5**
- 7.1 TSP bitmask $O(n^2 2^n)$, $n \leq 20$ . . . 5
- 7.2 SOS DP (sum over subsets/supersets) . . . 5
- 7.3 DP phân công (assignment) $O(n2^n)$ . . . 5
- 7.4 Meet-in-the-middle ($n \leq 40$) . . . 6

8 Luồng cực đại **6**
- 8.1 Dinic $O(E\sqrt{V}V?)$ đủ cho 10^4 đỉnh . . . 6
- 8.2 Hopcroft–Karp (ghép cặp cực đại bipartite) . . . 6
- 8.3 Min-Cost Max-Flow (potentials + Dijkstra) . . . 6

9 Local Search: Hill Climbing **6**
9.1 Hill Climbing (tô màu / lập lịch / TSP) . . . 6

10 Phụ lục: Số học và tổ hợp hay dùng **7**
10.1 Lũy thừa mod, nghịch đảo, nCr . . . 7
10.2 Đại số modulo . . . 7
10.3 Sàng tuyến tính + phân tích thừa số . . . 7
10.4 Euclid mở rộng . . . 7

1. BFS và các dạng đồ thị khó

Phần này gồm: BFS lưới có truy vết, BFS 0–1, Dijkstra heap, BFS đa nguồn, DSU + Kruskal, LCA.

1.1 BFS ma trận có truy vết đường đi

Ý tưởng: đánh dấu hướng đi tới mỗi ô, sau đó lội ngược từ B về A (bài toán mê cung, CSES 1193). Bảng `mask` lưu ký tự hướng đã dùng để tới ô đó.

```
// BFS luoi n x m, A -> B, vat can '#'. Tra ve chuoi duong L/R/U/D
string bfs_grid(vector<string>& a){
  int n=a.size(), m=a[0].size();
  pair<int,int> s{-1,-1}, t{-1,-1};
  for(int i=0;i<n;i++) for(int j=0;j<m;j++){
    if(a[i][j]=='A') s={i,j};
    if(a[i][j]=='B') t={i,j};
  }
  vector<string> mask(n, string(m,'?'));
  queue<pair<int,int>> q;
  q.push(s); mask[s.first][s.second]='S';
  int dx[4]={0,0,-1,1}; int dy[4]={-1,1,0,0};
  char dc[4]={'L','R','U','D'};
  bool found=false;
  while(!q.empty() && !found){
    auto [x,y]=q.front(); q.pop();
    for(int k=0;k<4;k++){
      int nx=x+dx[k], ny=y+dy[k];
      if(nx<0||nx>=n||ny<0||ny>=m) continue;
      if(a[nx][ny]=='#'||mask[nx][ny]!='?') continue;
      mask[nx][ny]=dc[k];
      q.push({nx,ny});
      if(nx==t.first&&ny==t.second){found=true;break;}
    }
  }
  if(!found) return "";
  string ans="";
  int x=t.first,y=t.second;
  unordered_map<char,pair<int,int>> d={
    {'L',{0,-1}},{'R',{0,1}},{'U',{-1,0}},{'D',{1,0}}};
  while(mask[x][y]!='S'){
    char c=mask[x][y]; ans+=c;
    x-=d[c].first; y-=d[c].second;
  }
  reverse(ans.begin(),ans.end());
  return ans;
}
```

1.2 BFS 0–1 (cạnh trọng số 0/1)

Dùng `deque`. Bài toán biến đổi tốn 0/1, lưới có chi phí vào ô.

```
vector<int> bfs01(vector<vector<pair<int,int>>>& adj,int s){
  int n=adj.size(); vector<int> d(n,INF);
  deque<int> dq; d[s]=0; dq.push_front(s);
  while(!dq.empty()){
    int u=dq.front(); dq.pop_front();
    for(auto [v,w]:adj[u]){
      if(d[v]>d[u]+w){ d[v]=d[u]+w;
        if(w==0) dq.push_front(v); else dq.push_back(v);
      }
    }
  }
  return d;
}
```

1.3 Dijkstra heap + dừng sớm

```
vector<ll> dijkstra(vector<vector<pair<int,ll>>>& adj,int s){
  int n=adj.size(); vector<ll> d(n,INF64);
  priority_queue<pair<ll,int>,vector<pair<ll,int>>,greater<>> pq;
  d[s]=0; pq.push({0,s});
  while(!pq.empty()){
    auto [du,u]=pq.top(); pq.pop();
    if(du!=d[u]) continue;
    for(auto [v,w]:adj[u])
      if(d[v]>du+w){d[v]=du+w; pq.push({d[v],v});}
  }
  return d;
}
```

1.4 BFS đa nguồn

Đẩy mọi nguồn vào queue với `dist=0`. Dùng cho: lan truyền lửa/lây nhiễm, khoảng cách tới ô đặc biệt gần nhất, Voronoi trên lưới.

1.5 DSU + Kruskal (khó ở nén + rollback tư duy)

```
struct DSU{
  vector<int> p,r;
  DSU(int n=0){init(n);}
  void init(int n){p.resize(n);r.assign(n,0);iota(p.begin(),p.end(),0);}
  int find(int x){return p[x]==x?x:p[x]=find(p[x]);}
  bool unite(int a,int b){
    a=find(a);b=find(b); if(a==b) return false;
    if(r[a]<r[b]) swap(a,b);
    p[b]=a; if(r[a]==r[b]) r[a]++; return true;
  }
};
```

1.6 LCA Binary Lifting

Tiền xử lý $O(n \log n)$, truy vấn $O(\log n)$. Cần cho bài đường đi + max/min cạnh.

```
const int LOG=20;
int n; vector<vector<int>> up; vector<int> dep;
```

```
vector<vector<pair<int,int>>> tree;
void dfs_lca(int u,int p){
  up[u][0]=p;
  for(int k=1;k<LOG;k++) up[u][k]=up[up[u][k-1]][k-1];
  for(auto [v,w]:tree[u]) if(v!=p){dep[v]=dep[u]+1;dfs_lca(v,u);}
}
int lca(int a,int b){
  if(dep[a]<dep[b]) swap(a,b);
  int d=dep[a]-dep[b];
  for(int k=0;k<LOG;k++) if(d>>k&1) a=up[a][k];
  if(a==b) return a;
  for(int k=LOG-1;k>=0;k--) if(up[a][k]!=up[b][k]){a=up[a][k];b=up[b][k];}
  return up[a][0];
}
```

2. Fenwick Tree (BIT)

Nhẹ hơn Segment Tree, code ngắn, ít bug. Nhớ: BIT gốc chỉ làm prefix; muốn đoạn phải biến đổi.

2.1 BIT cộng điểm – hỏi prefix/đoạn

```
struct BIT{
  int n; vector<ll> bit;
  BIT(int n=0):n(n),bit(n+1,0){}
  void add(int idx,ll val){ // 0-indexed
    for(int i=idx+1;i<=n;i+=i&-i) bit[i]+=val;
  }
  ll sumPrefix(int idx){ // sum [0..idx], idx<0 -> 0
    ll s=0;
    for(int i=idx+1;i>0;i-=i&-i) s+=bit[i];
    return s;
  }
  ll rangeSum(int l,int r){
    if(l>r) return 0;
    return sumPrefix(r)-(l?sumPrefix(l-1):0);
  }
};
```

2.2 BIT range-add – point-query và range-add – range-sum

Cộng v vào $[l,r]$: `add(l,v); add(r+1,-v)` rồi hỏi điểm bằng prefix. Muốn hỏi tổng đoạn sau khi cộng đoạn: dùng 2 BIT.

```
struct BITRange{
  int n; BIT b1,b2;
  BITRange(int n=0):n(n),b1(n+1),b2(n+1){}
  void _add(BIT& b,int idx,ll v){ if(idx<=n) b.add(idx,v); }
  // cong v vao [l,r]
  void rangeAdd(int l,int r,ll v){
    _add(b1,l,v); _add(b1,r+1,-v);
    _add(b2,l,v*(l-1)); _add(b2,r+1,-v*r);
  }
  ll prefixSum(int idx){ // tong [0..idx]
    return b1.sumPrefix(idx)*idx - b2.sumPrefix(idx);
  }
  ll rangeSum(int l,int r){ return prefixSum(r)-(l?prefixSum(l-1):0); }
};
```

2.3 BIT tìm chỉ số có prefix $\geq k$ (order statistics)

Dùng cho: xóa phần tử thứ k, Hotel Queries bản đơn giản, phân phối kẹo. $k \geq 1$.

```
// bit: BIT tan suat (0/1 hoac dem). Tim idx nho nhat sao cho prefix>=k
int kth(BIT& bit,int k){
  int idx=0;
  int pw=1; while((pw<<1)<=bit.n) pw<<=1;
  for(int d=pw;d>0;d>>=1){
    int nxt=idx+d;
    if(nxt<=bit.n && bit.bit[nxt]<k){k-=bit.bit[nxt]; idx=nxt;}
  }
  return idx; // 0-indexed: idx (vi bit 1-indexed) -> dap an 0-based = idx
}
```

2.4 BIT 2D (dạng khó)

Cộng điểm (x,y), hỏi tổng hình chữ nhật. $n,m \leq 2000$: $O(\log n \log m)$.

```
struct BIT2D{
  int n,m; vector<vector<ll>> bit;
  BIT2D(int n=0,int m=0):n(n),m(m),bit(n+1,vector<ll>(m+1,0)){}
  void add(int x,int y,ll v){
    for(int i=x+1;i<=n;i+=i&-i) for(int j=y+1;j<=m;j+=j&-j) bit[i][j]+=v;
  }
  ll sum(int x,int y){
    ll s=0;
    for(int i=x+1;i>0;i-=i&-i) for(int j=y+1;j>0;j-=j&-j) s+=bit[i][j];
    return s;
  }
  ll rect(int x1,int y1,int x2,int y2){
    return sum(x2,y2)-sum(x1-1,y2)-sum(x2,y1-1)+sum(x1-1,y1-1);
  }
};
```

3. Segment Tree

Khi BIT không đủ (max/min, tìm vị trí đầu tiên $\geq v$, gán đoạn). Cài đặt đệ quy với `smt[4*n]` gồm `build/query/update`.

3.1 SegTree lazy cộng – hỏi max/sum

Đoạn code dưới dùng cho range-add + range-max. Muốn sum thì đổi `node=max` thành `sum` và nhân lazy với độ dài.

```
struct SegLazy{
  int n; vector<ll> st,lz;
  SegLazy(vector<ll>& a){n=a.size();st.assign(4*n,0);lz.assign(4*n,0);build(0,0,n-1,a);}
  void build(int i,int l,int r,vector<ll>& a){
    if(l==r){st[i]=a[l];return;}
    int m=(l+r)/2; build(2*i+1,l,m,a); build(2*i+2,m+1,r,a);
    st[i]=max(st[2*i+1],st[2*i+2]);
  }
  void push(int i){
    if(lz[i]!=0){
      for(int k:{2*i+1,2*i+2}){st[k]+=lz[i]; lz[k]+=lz[i];}
      lz[i]=0;
    }
  }
  void add(int i,int l,int r,int ql,int qr,ll v){
    if(qr<l||r<ql) return;
    if(ql<=l&&r<=qr){st[i]+=v; lz[i]+=v; return;}
    push(i); int m=(l+r)/2;
    add(2*i+1,l,m,ql,qr,v); add(2*i+2,m+1,r,ql,qr,v);
    st[i]=max(st[2*i+1],st[2*i+2]);
  }
  ll query(int i,int l,int r,int ql,int qr){
    if(qr<l||r<ql) return -INF64;
    if(ql<=l&&r<=qr) return st[i];
    push(i); int m=(l+r)/2;
    return max(query(2*i+1,l,m,ql,qr),query(2*i+2,m+1,r,ql,qr));
  }
```

```
  void add(int l,int r,ll v){add(0,0,n-1,l,r,v);}
  ll query(int l,int r){return query(0,0,n-1,l,r);}
};
```

3.2 Tìm phòng đầu tiên đủ chỗ (Hotel Queries)

Mỗi khách cần phòng có sức chứa $\geq p$, chọn phòng trái nhất, trừ đi và cập nhật (bài toán Hotel Queries, CSES 1143). Lặp m lần $O((n+m)\log n)$.

```
void build_mx(vector<ll>& smt,vector<ll>& a,int i,int l,int r){
  if(l==r){smt[i]=a[l];return;}
  int m=(l+r)/2; build_mx(smt,a,2*i+1,l,m); build_mx(smt,a,2*i+2,m+1,r);
  smt[i]=max(smt[2*i+1],smt[2*i+2]);
}
int find_first(vector<ll>& smt,int i,int l,int r,ll val){
  if(smt[i]<val) return -1;
  if(l==r) return l;
  int m=(l+r)/2;
  if(smt[2*i+1]>=val) return find_first(smt,2*i+1,l,m,val);
  return find_first(smt,2*i+2,m+1,r,val);
}
void point_update(vector<ll>& smt,int i,int l,int r,int pos,ll v){
  if(l==r){smt[i]=v;return;}
  int m=(l+r)/2;
  if(pos<=m) point_update(smt,2*i+1,l,m,pos,v);
  else point_update(smt,2*i+2,m+1,r,pos,v);
  smt[i]=max(smt[2*i+1],smt[2*i+2]);
}
```

3.3 SegTree lặp (iterative) cho sum/min

Ngắn, nhanh, ít tràn stack. Phù hợp khi n tới $2 \cdot 10^5$ và query dày.

```
struct SegIter{
  int n; vector<ll> t; ll NEUT=0;
  SegIter(vector<ll>& a){
    n=1; while(n<(int)a.size()) n*=2;
    t.assign(2*n,NEUT);
    for(int i=0;i<(int)a.size();i++) t[n+i]=a[i];
    for(int i=n-1;i>0;i--) t[i]=t[2*i]+t[2*i+1];
  }
  void pointSet(int p,ll v){ p+=n; t[p]=v; for(p/=2;p;p/=2) t[p]=t[2*p]+t[2*p+1]; }
  ll rangeSum(int l,int r){ // [l,r]
    l+=n; r+=n; ll s=0;
    while(l<=r){ if(l%2==1) s+=t[l++]; if(r%2==0) s+=t[r--]; l/=2; r/=2; }
    return s;
  }
};
```

Muốn min/max: đổi `NEUT` thành `INF / -INF` và phép hợp thành `min/max`.

4. Trie

4.1 Trie chuỗi thường (đếm prefix)

Dùng mảng `next[26]` khi bảng chữ cái nhỏ; dùng `map` khi ký tự rộng. Nhớ khởi tạo `-1`.

```
struct Trie{
  vector<array<int,26>> nx; vector<int> cnt,end_;
  Trie(){nx.push_back({});nx[0].fill(-1);cnt={0};end_={0};}
  void insert(const string& s){
    int u=0; cnt[0]++;
    for(char ch:s){int c=ch-'a';
      if(nx[u][c]==-1){nx[u][c]=nx.size();nx.push_back({});nx.back().fill(-1);cnt.pb(0);end_.pb(0);}
      u=nx[u][c]; cnt[u]++;
    }
    end_[u]++;
  }
  int countPrefix(const string& p){
    int u=0; for(char ch:p){int c=ch-'a'; if(nx[u][c]==-1) return 0; u=nx[u][c];}
    return cnt[u];
  }
};
```

4.2 Binary Trie – XOR lớn nhất (dạng khó)

Chèn số, hỏi max xor với x. Dùng cho cặp xor max, truy vấn prefix xor.

```
struct BinTrie{
  struct Node{int ch[2];int cnt;Node(){ch[0]=ch[1]=-1;cnt=0;}};
  vector<Node> t; BinTrie(){t.emplace_back();}
  void insert(int x){
    int u=0; t[u].cnt++;
    for(int b=30;b>=0;b--){int bit=(x>>b)&1;
      if(t[u].ch[bit]==-1){t[u].ch[bit]=t.size();t.emplace_back();}
      u=t[u].ch[bit]; t[u].cnt++;
    }
  }
  int maxXor(int x){
    int u=0,ans=0;
    for(int b=30;b>=0;b--){int bit=(x>>b)&1,want=bit^1;
      if(t[u].ch[want]!=-1){ans|=1<<b;u=t[u].ch[want];}
      else u=t[u].ch[bit];
    }
    return ans;
  }
};
```

4.3 Aho–Corasick (Trie + KMP tập mẫu)

Kết hợp Trie và KMP để đếm số mẫu xuất hiện trong văn bản. $O(|text| + \sum |pat|)$.

```
struct Aho{
  struct Node{array<int,26> nx;int link;ll out;Node(){nx.fill(-1);link=0;out=0;}};
  vector<Node> t; Aho(){t.emplace_back();}
  void add(const string& s){int u=0;for(char ch:s){int c=ch-'a';
    if(t[u].nx[c]==-1){t[u].nx[c]=t.size();t.emplace_back();}u=t[u].nx[c];}t[u].out++;}
  void build(){
    queue<int> q;
    for(int c=0;c<26;c++){int v=t[0].nx[c];
      if(v!=-1){t[v].link=0;q.push(v);} else t[0].nx[c]=0;}
    while(!q.empty()){int u=q.front();q.pop();
      t[u].out+=t[t[u].link].out;
      for(int c=0;c<26;c++){int v=t[u].nx[c];
        if(v!=-1){t[v].link=t[t[u].link].nx[c];q.push(v);}
        else t[u].nx[c]=t[t[u].link].nx[c];}
    }
  }
  ll match(const string& s){int u=0;ll ans=0;for(char ch:s){u=t[u].nx[ch-'a'];ans+=t[u].out;}return ans
    ;}
};
```

5. KMP, Z, Hashing

5.1 Hàm prefix π và KMP tìm mẫu

```
vector<int> prefixFunc(const string& s){
  int n=s.size(); vector<int> pi(n,0);
  for(int i=1;i<n;i++){int j=pi[i-1];
    while(j>0&&s[i]!=s[j]) j=pi[j-1];
```

```
    if(s[i]==s[j]) j++; pi[i]=j;
  }
  return pi;
}
vector<int> kmp(const string& t,const string& p){
  string s=p+'#'+t; vector<int> pi=prefixFunc(s);
  vector<int> occ; int m=p.size();
  for(int i=m+1;i<(int)s.size();i++) if(pi[i]==m) occ.pb(i-2*m);
  return occ; // vi tri bat dau trong t
}
```

5.2 Hàm Z

`z[i]` = độ dài tiền tố khớp tại i. Dùng tìm mẫu, nén chuỗi, khớp cyclic shift.

```
vector<int> zFunc(const string& s){
  int n=s.size(); vector<int> z(n,0); int l=0,r=0;
  for(int i=1;i<n;i++){
    if(i<=r) z[i]=min(r-i+1,z[i-l]);
    while(i+z[i]<n&&s[z[i]]==s[i+z[i]]) z[i]++;
    if(i+z[i]-1>r){l=i;r=i+z[i]-1;}
  }
  return z;
}
```

5.3 Rolling Hash kép

Khi KMP/Z không đủ (so sánh chuỗi con bất kỳ, palindrome). Dùng 2 mod.

```
const int HMOD1=1000000007,HMOD2=1000000009,HBASE=9113823%100000+7;
struct HashStr{
  string s; vector<ll> p1,p2,h1,h2;
  HashStr(const string& str=""){init(str);}
  void init(const string& str){s=str;int n=s.size();
    p1.assign(n+1,1);p2.assign(n+1,1);h1.assign(n+1,0);h2.assign(n+1,0);
    for(int i=0;i<n;i++){p1[i+1]=p1[i]*HBASE%HMOD1;p2[i+1]=p2[i]*HBASE%HMOD2;
      h1[i+1]=(h1[i]*HBASE+s[i])%HMOD1;h2[i+1]=(h2[i]*HBASE+s[i])%HMOD2;}}
  pair<ll,ll> get(int l,int r){ // [l,r]
    ll a=(h1[r+1]-h1[l]*p1[r-l+1]%HMOD1+HMOD1)%HMOD1;
    ll b=(h2[r+1]-h2[l]*p2[r-l+1]%HMOD2+HMOD2)%HMOD2;
    return {a,b};
  }
};
```

5.4 Manacher (palindrome dài nhất $O(n)$)

Thuật toán tuyến tính tìm mọi bán kính palindrome lẻ và chẵn.

```
// d1[i]: ban kinh palindrome le tam i (so o le), d2[i]: chan tam giua i-1,i
pair<vector<int>,vector<int>> manacher(const string& s){
  int n=s.size(); vector<int> d1(n),d2(n);
  for(int i=0,l=0,r=-1;i<n;i++){int k=(i>r)?1:min(d1[l+r-i],r-i+1);
    while(i-k>=0&&i+k<n&&s[i-k]==s[i+k]) k++; d1[i]=k--;
    if(i+k>r){l=i-k;r=i+k;}}
  for(int i=0,l=0,r=-1;i<n;i++){int k=(i>r)?0:min(d2[l+r-i+1],r-i+1);
    while(i-k-1>=0&&i+k<n&&s[i-k-1]==s[i+k]) k++; d2[i]=k--;
    if(i+k>r){l=i-k-1;r=i+k;}}
  return {d1,d2};
}
```

6. DP chữ số (Digit DP)

Trạng thái gồm `pos, tight, started/leadingZero, sum/mod`. Đệ quy có memo khi `tight=0`.

```
string S; ll memo[20][200][2]; bool vis[20][200][2];
// dem so <= S co tong chu so == target (bo so 0? tu quy dinh)
ll dfs(int pos,int sum,bool tight,bool started,int target){
  if(pos==(int)S.size()) return (started&&sum==target)?1:0;
  if(!tight&&vis[pos][sum][started]) return memo[pos][sum][started];
  int lim=tight?S[pos]-'0':9; ll res=0;
  for(int d=0;d<=lim;d++){
    bool nstarted=started||(d!=0);
    int nsum=sum+(nstarted?d:0);
    if(nsum>target) continue;
    res+=dfs(pos+1,nsum,tight&&(d==lim),nstarted,target);
  }
  if(!tight){vis[pos][sum][started]=true; memo[pos][sum][started]=res;}
  return res;
}
```

Cách gọi: `S=to_string(R); memset(vis,0,sizeof vis); ans=dfs(0,0,true,false,target)`. Đếm đoạn $[L,R]$: `F(R)-F(L-1)`.

Ví dụ đếm số không chứa chữ số 4 và không chứa chuỗi 13:

```
ll dp2[20][2][2][2]; bool v2[20][2][2][2]; string T;
ll dfs2(int p,bool t,bool started,bool hasBad,int prevHas1){
  // don gian hoa: truyen prev la co phai so truoc la 1 khong
  if(p==(int)T.size()) return (!started||!hasBad)?1:0; // tuy de
  // ... lap d nhu tren, hasBad |= (d==4)||(prevHas1&&d==3&&started);
  return 0; // viet day du khi thi
}
```

7. DP Bitmask và các dạng khó

Phần này gồm: TSP Held–Karp, SOS DP, phân công, đếm Hamiltonian.

7.1 TSP bitmask $O(n^2 2^n)$, $n \le 20$

```
ll tsp(vector<vector<ll>>& c){
  int n=c.size(), N=1<<n; const ll INFLL=INF64;
  vector<vector<ll>> dp(N,vector<ll>(n,INFLL));
  dp[1][0]=0;
  for(int mask=1;mask<N;mask++) for(int u=0;u<n;u++){
    if(!(mask&1<<u)||dp[mask][u]==INFLL) continue;
    for(int v=0;v<n;v++) if(!(mask&1<<v))
      dp[mask|1<<v][v]=min(dp[mask|1<<v][v],dp[mask][u]+c[u][v]);
  }
  ll ans=INFLL;
  for(int u=1;u<n;u++) ans=min(ans,dp[N-1][u]+c[u][0]);
  return ans;
}
```

7.2 SOS DP (sum over subsets/supersets)

$F[mask] = \sum_{sub \subseteq mask} A[sub]$. $O(k2^k)$.

```
void sos(vector<ll>& f,int k){
  int N=1<<k;
  for(int b=0;b<k;b++) for(int mask=0;mask<N;mask++)
    if(mask&1<<b) f[mask]+=f[mask^(1<<b)];
}
```

Dùng cho: đếm cặp (a,b) sao cho $a\&b == 0$, bao đóng tập, DP trên mặt nạ tần suất.

7.3 DP phân công (assignment) $O(n2^n)$

`dp[mask]` = chi phí min khi đã gán `popcount(mask)` việc đầu cho tập người `mask`.

```
ll assignMin(vector<vector<ll>>& a){
```

```
  int n=a.size(),N=1<<n; vector<ll> dp(N,INF64); dp[0]=0;
  for(int mask=0;mask<N;mask++){
    int i=__builtin_popcount(mask); // viec thu i
    for(int j=0;j<n;j++) if(!(mask&1<<j))
      dp[mask|1<<j]=min(dp[mask|1<<j],dp[mask]+a[i][j]);
  }
  return dp[N-1];
}
```

7.4 Meet-in-the-middle ($n \le 40$)

Chia đôi, liệt kê $2^{n/2}$ tổng mỗi nửa, sort + two-pointer / binary search. Dùng cho subset-sum, knapsack lớn, xor min.

```
// vi du: dem so tap co tong == S
ll meetCount(vector<ll>& a,ll S){
  int n=a.size(),n1=n/2; vector<ll> L,R;
  for(int m=0;m<1<<n1;m++){ll s=0;for(int i=0;i<n1;i++)if(m>>i&1)s+=a[i];L.pb(s);}
  int n2=n-n1;
  for(int m=0;m<1<<n2;m++){ll s=0;for(int i=0;i<n2;i++)if(m>>i&1)s+=a[n1+i];R.pb(s);}
  sort(R.begin(),R.end()); ll ans=0;
  for(ll x:L) ans+=upper_bound(R.begin(),R.end(),S-x)-lower_bound(R.begin(),R.end(),S-x);
  return ans;
}
```

8. Luồng cực đại

8.1 Dinic $O(E\sqrt{V}?)$ đủ cho 10^4 đỉnh

Mẫu cài đặt chuẩn: BFS level + DFS blocking flow + current arc.

```
struct Dinic{
  struct E{int to;ll cap;int rev;};
  int n; vector<vector<E>> g; vector<int> lv,it;
  Dinic(int n=0):n(n),g(n),lv(n),it(n){}
  void addEdge(int u,int v,ll c){E a{v,c,(int)g[v].size()};E b{u,0,(int)g[u].size()};g[u].pb(a);g[v].pb(
      b);}
  bool bfs(int s,int t){
    fill(lv.begin(),lv.end(),-1); queue<int> q; lv[s]=0; q.push(s);
    while(!q.empty()){int u=q.front();q.pop();
      for(auto &e:g[u]) if(e.cap>0&&lv[e.to]<0){lv[e.to]=lv[u]+1;q.push(e.to);}}
    return lv[t]>=0;
  }
  ll dfs(int u,int t,ll f){
    if(u==t) return f;
    for(int &i=it[u];i<(int)g[u].size();i++){
      E &e=g[u][i];
      if(e.cap>0&&lv[u]<lv[e.to]){
        ll d=dfs(e.to,t,min(f,e.cap));
        if(d>0){e.cap-=d; g[e.to][e.rev].cap+=d; return d;}
      }
    }
    return 0;
  }
  ll maxflow(int s,int t){ll flow=0,INF_F=INF64;
    while(bfs(s,t)){fill(it.begin(),it.end(),0);ll f;
      while((f=dfs(s,t,INF_F))>0) flow+=f;}
    return flow;
  }
};
```

8.2 Hopcroft–Karp (ghép cặp cực đại bipartite)

$O(E\sqrt{V})$. Dùng khi Dinic quá chậm cho bipartite 10^5.

```
struct HopcroftKarp{
  int nL,nR; vector<vector<int>> adj; vector<int> dist,pu,pv;
  HopcroftKarp(int L,int R):nL(L),nR(R),adj(L),dist(L),pu(L,-1),pv(R,-1){}
  void add(int u,int v){adj[u].pb(v);}
  bool bfs(){
    queue<int> q;
    for(int u=0;u<nL;u++){ if(pu[u]==-1){dist[u]=0;q.push(u);} else dist[u]=-1;}
    bool found=false;
    while(!q.empty()){int u=q.front();q.pop();
      for(int v:adj[u]){int w=pv[v];
        if(w!=-1&&dist[w]==-1){dist[w]=dist[u]+1;q.push(w);}
        if(w==-1) found=true;}}
    return found;
  }
  bool dfs(int u){
    for(int v:adj[u]){int w=pv[v];
      if(w==-1||(dist[w]==dist[u]+1&&dfs(w))){pu[u]=v;pv[v]=u;return true;}}
    dist[u]=-1; return false;
  }
  int maxmatch(){int m=0;
    while(bfs()){for(int u=0;u<nL;u++) if(pu[u]==-1&&dfs(u)) m++;}
    return m;
  }
};
```

8.3 Min-Cost Max-Flow (potentials + Dijkstra)

Dùng cho gán chi phí min, vận chuyển. $n \le 500$.

```
struct MCMF{
  struct E{int to,rev;ll cap,cost;};
  int n; vector<vector<E>> g;
  MCMF(int n=0):n(n),g(n){}
  void addEdge(int u,int v,ll cap,ll cost){
    E a{v,(int)g[v].size(),cap,cost},b{u,(int)g[u].size(),0,-cost};
    g[u].pb(a);g[v].pb(b);}
  pair<ll,ll> minCost(int s,int t,ll maxf=INF64){
    ll flow=0,cost=0; vector<ll> h(n,0),d(n); vector<int> pv(n),pe(n);
    while(flow<maxf){
      priority_queue<pair<ll,int>,vector<pair<ll,int>>,greater<>> pq;
      fill(d.begin(),d.end(),INF64); d[s]=0; pq.push({0,s});
      while(!pq.empty()){auto [du,u]=pq.top();pq.pop();
        if(du!=d[u]) continue;
        for(int i=0;i<(int)g[u].size();i++){E &e=g[u][i];
          if(e.cap>0&&d[e.to]>du+e.cost+h[u]-h[e.to]){
            d[e.to]=du+e.cost+h[u]-h[e.to];pv[e.to]=u;pe[e.to]=i;pq.push({d[e.to],e.to});}}}
      if(d[t]==INF64) break;
      for(int i=0;i<n;i++) if(d[i]<INF64) h[i]+=d[i];
      ll add=maxf-flow;
      for(int v=t;v!=s;v=pv[v]) add=min(add,g[pv[v]][pe[v]].cap);
      for(int v=t;v!=s;v=pv[v]){E &e=g[pv[v]][pe[v]];e.cap-=add;g[v][e.rev].cap+=add;}
      flow+=add; cost+=add*h[t];
    }
    return {flow,cost};
  }
};
```

9. Local Search: Hill Climbing

Dùng khi bài NP-khó cần điểm từng phần (TSP, sắp xếp, tô màu, lập lịch). Khung chung: biểu diễn nghiệm + hàm delta + vòng lặp có giới hạn giờ.

9.1 Hill Climbing (tô màu / lập lịch / TSP)

```cpp
// hill climbing co khoi dong lai (random restart)
template<class State,class Cost,class Neighbor>
State hillClimb(State s,Cost cost,Neighbor neigh,int iters=200000,int restarts=20){
  mt19937 r2(12345); State best=s; auto bestC=cost(s);
  for(int rs=0;rs<restarts;rs++){
    State cur=(rs==0?s:neigh(s,true)); auto curC=cost(cur);
    for(int it=0;it<iters;it++){
      State ns=neigh(cur,false); auto nc=cost(ns);
      if(nc<curC){cur=ns;curC=nc; if(nc<bestC){best=ns;bestC=nc;}}
      // tabu don gian: cam quay lai trang thai vua gap bang unordered_set hash
    }
  }
  return best;
}
```

10. Phụ lục: Số học và tổ hợp hay dùng

10.1 Lũy thừa mod, nghịch đảo, nCr

```cpp
const ll MOD=1e9+7;
ll modpow(ll a,ll e,ll mod=MOD){ll r=1;while(e){if(e&1)r=r*a%mod;a=a*a%mod;e>>=1;}return r;}
// nCr nho voi giai thua san (n<=2e6)
vector<ll> fact,invFact;
void initFact(int n){fact.assign(n+1,1);invFact.assign(n+1,1);
  for(int i=1;i<=n;i++) fact[i]=fact[i-1]*i%MOD;
  invFact[n]=modpow(fact[n],MOD-2); for(int i=n;i>0;i--) invFact[i-1]=invFact[i]*i%MOD;}
ll Cnk(int n,int k){if(k<0||k>n) return 0; return fact[n]*invFact[k]%MOD*invFact[n-k]%MOD;}
```

10.2 Đại số modulo

Mọi phép toán đưa về $[0, MOD)$. Nhớ $a-b$ có thể âm nên cộng MOD; $a \cdot b$ tràn 64-bit khi $MOD \sim 10^9$ thì dùng `__int128`. Chia modulo = nhân nghịch đảo (chỉ khi MOD nguyên tố và a không chia hết cho MOD).

```cpp
// MOD da dinh nghia o muc tren
inline ll addmod(ll a,ll b){ll s=a+b; if(s>=MOD) s-=MOD; return s;}
inline ll submod(ll a,ll b){ll s=a-b; if(s<0) s+=MOD; return s;}
inline ll mulmod(ll a,ll b){return (ll)((__int128)(a%MOD)*(b%MOD)%MOD);}
ll modpow(ll a,ll e,ll mod=MOD){ll r=1%mod; a%=mod; while(e){if(e&1)r=mulmod(r,a); a=mulmod(a,a); e
    >>=1;} return r;}
ll modInv(ll a){return modpow(a,MOD-2);} // MOD nguyen to
ll modDiv(ll a,ll b){return mulmod(a,modInv(b));}
// nghich dao 1..n tuyen tinh O(n) (MOD nguyen to)
vector<ll> buildInv(int n){vector<ll> inv(n+1); inv[1]=1;
  for(int i=2;i<=n;i++) inv[i]=MOD-MOD/i*inv[MOD%i]%MOD; return inv;}
// tong cap so nhan 1+a+...+a^{n-1} mod MOD
ll geomSum(ll a,ll n){
  if(n==0) return 0; if(n==1) return 1;
  if(n%2==0){ll h=geomSum(a,n/2); return mulmod(h,addmod(1,modpow(a,n/2)));}
  return addmod(geomSum(a,n-1),modpow(a,n-1));
}
ll Pnk(int n,int k){if(k<0||k>n) return 0; ll r=1; for(int i=0;i<k;i++) r=mulmod(r,n-i); return r;}
```

10.3 Sàng tuyến tính + phân tích thừa số

```cpp
vector<int> primes,lp;
void sieve(int N){lp.assign(N+1,0);
  for(int i=2;i<=N;i++){if(lp[i]==0){lp[i]=i;primes.pb(i);}
    for(int p:primes){if(p>lp[i]||1LL*i*p>N) break; lp[i*p]=p;}}}
```

10.4 Euclid mở rộng

```cpp
ll exgcd(ll a,ll b,ll &x,ll &y){
  if(b==0){x=1;y=0;return a;}
  ll x1,y1; ll g=exgcd(b,a%b,x1,y1); x=y1; y=x1-y1*(a/b); return g;
}
```